CS108 Mini Game Hub: Final Report

Sathwik & Manohar [1]

April 29, 2026

[1] https://www.pygame.org/wiki/tutorials & https://docs.python.org/3/

Contents

0.1 Intro . . . 2
0.2 External Libraries . . . 2
0.3 Features implemented . . . 3
0.3.1 Authentication (main.sh) . . . 3
0.3.2 Game Menu (game.py) . . . 3
0.3.3 Games . . . 3
0.3.4 Technical Requirements . . . 3
0.3.5 Analytics . . . 3
0.4 Explanation of Implimentation of code . . . 4
0.4.1 Authentication System (`main.sh`) . . . 4
0.4.2 Game Hub and Core Engine (`game.py` & Base Class) . . . 4
0.5 Game Logic Implementations . . . 4
0.5.1 Tic-Tac-Toe (`tictactoe.py`) . . . 4
0.5.2 Connect 4 (`connect4.py`) . . . 4
0.5.3 Othello / Reversi (`Othello.py`) . . . 5
0.6 Leaderboard Management (`leaderboard.sh`) . . . 5
0.7 Hurdles Faced and Resolutions . . . 5
0.8 Future Improvements (With 10 Additional Hours) . . . 6

0.1 Intro

In this report you will know about how we used libraries , A description of all features implemented , Hurdles faced and how we overcame them.

0.2 External Libraries

1. **Numpy** : NumPy is used solely to initialize the game board as a 2D array of zeros with np.zeros((n, n), dtype=int), where each cell stores 0 (empty), 1 (player 1), or 2 (player 2). The win condition logic then iterates over this matrix to count consecutive matching values across rows, columns, and diagonals.

2. **pygame** : Pygame serves as the entire graphics and interaction engine — it renders the game hub window, loads and scales background/game images, and handles mouse click events to launch individual games. It draws the board, animates win lines, and displays winner/tie text using fonts and surfaces.

3. **subprocess** : Subprocess is used to launch each individual game (tictactoe, othello, connect4) as separate Python processes, capture their return codes to determine the winner, run a bash leaderboard script, and trigger a plot script , it connects all components of the game hub together.

4. **sys** : Sys is used to accept command-line arguments (sys.argv) for player names at startup, and to cleanly terminate the program via sys.exit() when a player chooses not to continue after a game.

5. **time** : Time is used solely to fetch the current date via time.strftime"$\%Y-\%m-\%d$" when recording match results into the history CSV file.

6. **pathlib** : Pathlib is used solely to define the history CSV file path via Path("history.csv"), ensuring cross-platform file path handling when appending match results to the history file.

7. **matplotlib** : For showing A bar chart of the Top 5 Players by total win count, A pie chart of the Most Played Games by frequency in history.csv .

0.3 Features implemented

0.3.1 Authentication (main.sh)

- Two-player login with SHA-256 hashed passwords stored in users.tsv
- Registration for new users, with re-prompt on wrong password

0.3.2 Game Menu (game.py)

- Pygame GUI hub to select from 3 games, with both usernames passed as arguments

0.3.3 Games

- Tic-Tac-Toe on 10x10 board, 5-in-a-row to win
- Othello on 8x8 with disc flipping logic and turn skipping
- Connect Four on 7x7 with gravity-based coin dropping, 4-in-a-row to win

0.3.4 Technical Requirements

- All boards as NumPy arrays; win checks via array slicing, not manual loops
- Full Pygame GUI for every game, no terminal gameplay
- Base class with shared logic; each game inherits from it in a separate file.

0.3.5 Analytics

- Append winner, loser, date, game to history.csv after each game
- Call leaderboard.sh to display wins/losses/ratio per player per game, sorted by a GUI-selected metric
- Matplotlib bar chart (top 5 players) and pie chart (most played games)

0.4 Explanation of Implimentation of code

0.4.1 Authentication System (`main.sh`)

The gateway to the game hub is a Bash-based authentication system that ensures only registered users can access the games.

- **User Management:** Utilizes a `users.tsv` file as a lightweight database to store usernames and SHA-256 hashed passwords.
- **Dual Authentication:** The `cond` function requires two distinct users to log in consecutively. If a user does not exist, they are seamlessly routed to a registration flow.
- **Session Launch:** Once two unique players are authenticated, the script launches the Python game environment (`game.py`), passing the authenticated usernames as command-line arguments.

0.4.2 Game Hub and Core Engine (`game.py` & Base Class)

The primary interface is a Pygame-based hub that acts as a centralized launcher and manager for the mini-games.

- **Generic Base Class:** A shared player and board management system provides an $n \times n$ NumPy array for grid representation. Turn management avoids complex conditionals by using a mathematical toggle: subtracting the current turn from 3 (switching 1 to 2, and 2 to 1).
- **Main Menu Loop:** Initializes the display and waits for mouse click events on bounding rectangles associated with game icons. Selecting a game enters its specific, isolated game loop.
- **Endgame State:** Upon a win or tie, the hub displays a results screen and provides interactive UI elements that use `subprocess` calls to trigger external shell scripts for leaderboard generation.

0.5 Game Logic Implementations

0.5.1 Tic-Tac-Toe (`tictactoe.py`)

- **Board Mechanics:** Played on an expanded 10×10 NumPy grid mapped to screen rectangles.
- **Win Condition:** Requires 5-in-a-row to win. The logic converts the board to a boolean mask for the current player and uses NumPy slicing to check for overlapping sequences of five across all rows, columns, and both diagonals.

0.5.2 Connect 4 (`connect4.py`)

- **Board Mechanics:** Played on a standard 7×7 grid. The move logic simulates gravity by iterating through the selected column and placing the piece in the lowest available row index.
- **Win Condition:** Similar to Tic-Tac-Toe, it relies on boolean masks and NumPy slicing to detect 4-in-a-row patterns in all horizontal, vertical, and diagonal directions.

0.5.3 Othello / Reversi (`Othello.py`)

- **Valid Move Detection:** Uses an 8×8 board. The `validmoves` function scans empty cells and checks all eight cardinal and ordinal directions using a `checkfun` helper to see if placing a piece traps opponent pieces.
- **Piece Flipping:** The `makemove` function verifies legality, places the piece, and iterates backward along valid vectors to flip trapped opposing pieces.
- **Win Condition:** The game concludes when `wincond` determines neither player has any valid moves remaining. The winner is decided by a final board count.

0.6 Leaderboard Management (`leaderboard.sh`)

Game outcomes are persistently tracked and formatted using a dedicated Bash script.

- **Data Parsing:** Reads from a `history.csv` log. It uses `awk` to sanitize carriage returns, iterate through match data, and tally wins, losses, and ties for every user-game pair.
- **Metric Calculation:** Computes a win-loss ratio, safely handling edge cases (e.g., zero losses).
- **Dynamic Sorting:** Accepts a command-line argument (`wins`, `losses`, or `ratio`) to dynamically sort the parsed data structures before outputting them.
- **Formatting:** Passes the final sorted dataset through a secondary `awk` command to print a neatly aligned ASCII table for the user interface.

0.7 Hurdles Faced and Resolutions

Developing a cohesive Mini-Game Hub presented several technical and architectural challenges. Below are the primary hurdles encountered and the strategies used to overcome them:

- **Designing a Flexible Generic Game Class:** Initially, creating a single base class that could accommodate the varied logic, board sizes, and win conditions of different games (e.g., a 10×10 Tic-Tac-Toe versus other mini-games) was difficult. *Resolution:* We overcame this by leveraging object-oriented design principles, specifically using abstract classes and generics. By defining a generic game loop with abstract methods like `initializeBoard()`, `processTurn()`, and `checkWinCondition()`, each specific game could inherit the core structure while fully customizing its internal logic.
- **Secure Authentication and Session State:** Managing user sessions, ensuring that a user remains logged in across multiple game instances, and securely handling passwords posed a significant hurdle. *Resolution:* We implemented a centralized Authentication Manager using the Singleton pattern to ensure only one session state exists at a time. To secure user data, we utilized basic hashing algorithms for password storage rather than saving plain text, referencing standard security practices.
- **Dynamic Leaderboard Sorting:** The leaderboard needed to dynamically sort players based on varying metrics (e.g., total wins, win ratio, or high scores) across multiple different games. *Resolution:* We tackled this by implementing custom Comparator interfaces. This allowed the leaderboard module to fetch raw player statistics and sort the data structures on the fly depending on the specific game context requested by the user.

0.8 Future Improvements (With 10 Additional Hours)

Given an additional 10 hours of development time, we would focus on expanding the project's scalability and user experience in the following ways:

1. **Graphical User Interface (GUI):** Transition the current command-line interface to a fully interactive GUI. This would involve utilizing libraries to create clickable game boards, visual login screens, and formatted leaderboard tables, vastly improving user engagement.
2. **Networked Multiplayer Capabilities:** Implement socket programming to allow two users on different machines to play against each other in real-time, moving beyond local hot-seat multiplayer.
3. **Persistent Database Integration:** Replace the current file-based data storage system with a relational database (such as SQLite or PostgreSQL) to manage user accounts and leaderboard statistics more robustly and efficiently.

Bibliography

[1] Shinners, P., et al. (2023). *Pygame: Python Game Development*. Available at: https://www.pygame.org/ (Accessed: 25 October 2023).

[2] Harris, C. R., et al. (2020). *Array programming with NumPy*. Nature, 585(7825), 357-362. Available at: https://numpy.org/

[3] Fox, B., and Ramey, C. (2022). *GNU Bash Reference Manual*. Free Software Foundation. Available at: https://www.gnu.org/software/bash/manual/